

● Autonomous offensive security



An autonomous security *researcher.*

It learns how an application is meant to work, finds where its own rules break, and proves it, for modern web apps and APIs.

THE PROBLEM

The highest impact bugs require *understanding* the app.

- **Scanners test a fixed list of known vulnerability types.** They have no idea what the application is actually supposed to do.
- **The most damaging issues live in that gap** , like broken access control, business logic abuse, and privilege escalation. They depend entirely on context.
- **Finding them means human researchers** , scarce, expensive, and impossible to run continuously across everything you ship.

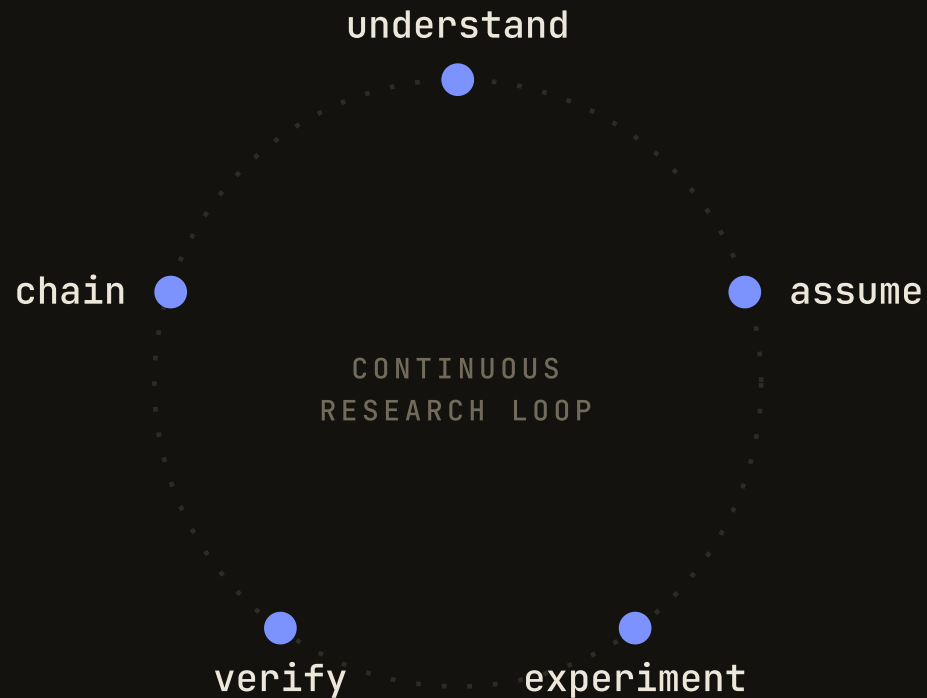
THE IDEA

Test the *assumptions*, not the payloads.

A strong researcher models how an application is meant to behave, then designs the one experiment that should be forbidden, and watches what happens.

We are building an autonomous agent that works the same way: understand the app, form the assumptions it makes, try to break each one, and verify the result.

A researcher's loop, run on its *own*.



- **Reasoning engine** builds a live model of the app, forms its assumptions, and designs the experiments that would break them.
- **Verification** confirms each finding with real, repeatable evidence, a proven contradiction, not a guess.
- **Chaining** feeds every confirmed result back in as the next lead, the way an attacker actually works.

WHAT'S DIFFERENT

Built to *reason*, designed to be trusted.

01

Reasons, doesn't scan

Works from assumptions about your app, not a fixed checklist of vulnerability types.

02

Verified, not guessed

Every finding rests on repeatable evidence from a real request, so there is little to triage.

03

Findings become tests

Each confirmed issue is a repeatable proof that can check the fix again over time.

04

Depth and breadth

Covers the surface while chaining single findings into deeper, multi step attacks.

05

APIs and modern apps

Reads API specifications and drives a real browser for apps heavy on JavaScript.

06

Efficient by design

Spends costly reasoning only where it pays off, so coverage grows without runaway cost.

EARLY SIGNAL

An early run gave us reason to *keep going.*

- A frontier model drove the full methodology autonomously, with no human in the loop.
- It identified and verified real access control and injection issues in test applications it had not seen before.

This is an early, encouraging indication of the approach, not a benchmark. Validation at scale is still ahead of us.

WHERE WE ARE TODAY

An honest *snapshot*.

BUILT AND WORKING

The loop runs end to end

An autonomous research loop runs against live test applications today, understanding, experimenting, and verifying on its own, across web apps and APIs.

NOT YET

Where we're still early

Early stage, small team. No external benchmark or third party validation yet. The deterministic verification layer and chaining across runs are being expanded.

Current focus: making the autonomous loop reliable, measurable, and consistent before opening it up.

WHY NOW

The pieces just came *together*.

- **Models can finally reason and use tools** across many steps, enough to carry a real research method, not just answer a prompt.
- **Demand far outpaces supply** of skilled security researchers, and every team ships faster than it can be tested.
- **The market is shifting to continuous, verified testing**, away from point in time scans toward proof that holds over time.

ROADMAP

From a reliable loop to *continuous* assurance.

NOW

Reliable autonomy

Make the research loop consistent and measurable; strengthen coverage and depth on web apps and APIs.

NEXT

Stronger verification

Expand deterministic verification and carry confirmed results forward across a run for deeper chaining.

LATER

Continuous testing

Test again on every change and turn each finding into a regression check wired into the release pipeline.

WHERE WE'RE HEADED

Point it at an app. *Trust the findings.*

We're building toward an autonomous researcher you can run continuously and rely on. We're looking for early design partners to shape it with us.

unboundcompute.com